L43 – Duplicate File Finder

Student: PRIYANSH SHAH | **Roll:** E41 | **Division:** G | **Enrollment:** 24004501210390

Project: L43 – Duplicate File Finder

Course: Linux Lab / Operating Systems Systems Programming (Sem-5)

---

1. Problem Statement

In multi-user operating systems and local development environments, folders accumulate duplicate files due to repeated downloads, manual backups, and project copies. These duplicates waste expensive disk space and cause confusion. The goal of this project is to build an interactive, non-destructive duplicate file management utility that:

1. Calculates cryptographic MD5 hashes for all files in a target directory.
2. Groups redundant files by identical checksums using Linux pipeline filters (`sort` , `uniq -w 32`).
3. Safely isolates duplicate copies into a designated `_duplicates/` folder without deleting them.
4. Quantifies reclaimed storage space using `du` .
5. Provides a reliable one-step undo capability to restore relocated files.
6. Automates background audit scans on a recurring schedule via `crontab` .

---

2. Who Will Use It

- System administrators who need to reclaim storage across home directories safely without data loss.
- Students and software developers organizing messy project workspaces, repositories, and download folders.
- Backup operators who want scheduled, non-intrusive duplicate audits before archiving.

---

3. Folder Structure

```
~/linux_project/dupfinder/
├── main.sh                 # Interactive menu and CLI argument dispatcher
├── README.md               # Project documentation, data format & reflections
├── populate_testdata.sh    # Script to populate >=20 realistic sample files
├── lib/
│   └── functions.sh        # Implementation of all 5 features, logging & cron
├── data/
│   └── testfiles/          # Working dataset containing 26 files (including 5 duplicate groups)
│       └── _duplicates/    # Temporary isolation folder created when moving duplicates
├── reports/                # Output audit reports generated by Feature 1/2 and cron tasks
├── backups/                # Timestamped .tar.gz archives created by option 7 / --backup
└── logs/
    ├── activity.log        # Timestamped log of user interactions and operations
    ├── cron.log            # Standard output/error capture from cron executions
    └── moved_files.txt     # Manifest tracking relocated files for safe undo
```

---

4. Test Data Structure & Column Explanation

The test dataset resides in `data/testfiles/` and contains **26 files** (exceeding the 20-file requirement). It includes structured tabular data (`.csv`), system configurations (`.cfg`), structured metrics (`.json`), system logs (`.log`), and notes (`.txt` , `.md`).

Tabular CSV Data Column Explanation:

1. **`students_sem5.csv`** (has duplicate copies `students_sem5_backup.csv` and `students_sem5_copy.csv`):

- `StudentID` : Unique student identification code (e.g., `E41`).
- `StudentName` : Full name of the enrolled student.
- `Course` : Academic course / laboratory module (e.g., `Linux Lab`).
- `Marks` : Internal assessment score out of 100.
- `Grade` : Performance letter grade (e.g., `A+` , `A` , `B+`).

1. **`grades_midterm.csv`** :

- `RollNo` : Class roll number.
- `Subject` : Subject name (e.g., `Operating Systems` , `Database Management`).
- `MidtermMarks` : Score achieved in the midterm test.
- `MaxMarks` : Total achievable marks (30).

1. **`lab_attendance.csv`** :

- `Date` : Date of the lab session (`YYYY-MM-DD`).

- `RollNo` : Student roll number.
- `StudentName` : Student name.
- `Status` : Attendance record (`Present` / `Absent`).

1. `faculty_contacts.csv` :

- `FacultyID` : Faculty employee identifier.
- `FacultyName` : Name and title of the professor.
- `Department` : Department affiliation.
- `Email` : Academic email address.
- `OfficeRoom` : Campus cabin room number.

Summary of Intentional Duplicate Groups in Test Data:

- **Group 1 (Triplicate CSV)**: `students_sem5.csv` , `students_sem5_backup.csv` , `students_sem5_copy.csv`
- **Group 2 (Duplicate CFG)**: `database_config.cfg` , `database_config_dup.cfg`
- **Group 3 (Duplicate TXT)**: `project_notes.txt` , `project_notes_copy.txt`
- **Group 4 (Duplicate LOG)**: `server_access.log` , `server_access_old.log`
- **Group 5 (Duplicate JSON)**: `system_metrics.json` , `system_metrics_copy.json`

---

5. Features

1. **Compute md5sum for all files**: Computes 128-bit cryptographic MD5 digests for each regular file in the target folder, formatted with size and filenames.
2. **Group duplicates**: Uses `find ... -exec md5sum {} + | sort | uniq -w 32 -D` to isolate identical 32-hex character hashes and display redundant copies grouped together.
3. **Move duplicates to `_duplicates/` (no delete)**: Retains the primary original in `data/testfiles/` and moves extra copies to `data/testfiles/_duplicates/` . Every move is recorded in `logs/moved_files.txt` .
4. **Report space saved (`du`)**: Inspects `_duplicates/` using `du -sh` and `ls -lh` to report total storage recovered.
5. **Undo by moving them back**: Reads `logs/moved_files.txt` and moves files back to their exact original locations, ensuring zero data loss and clean state rollback.
6. **View data**: Lists all active files and count in `data/testfiles/` .
7. **Backup data (`tar.gz`)**: Archives `data/` into `backups/data_YYYYMMDD_HHMMSS.tar.gz` .
8. **View activity log**: Displays the last 15 actions logged to `logs/activity.log` .

---

6. How to Run

Ensure executable permissions are set:

```
chmod +x main.sh lib/functions.sh populate_testdata.sh
```

Interactive Menu Mode:

```
cd ~/linux_project/dupfinder
./main.sh
```

Command-Line Arguments:

```
# Display help and usage instructions ($1 == --help)
./main.sh --help

# Create immediate tar.gz archive of data/ ($1 == --backup)
./main.sh --backup

# Run scheduled duplicate audit in report-only mode ($1 == --cron)
./main.sh --cron
```

---

7. Crontab Task

Requirement: Every Sunday at 3 AM, run a duplicate scan in report-only mode and save the report.

Crontab Entry:

```
0 3 * * 0 /bin/bash $HOME/linux_project/dupfinder/main.sh --cron >> $HOME/linux_project/dupfinder/lc
```

Breakdown of the 5 Time Fields:

Field	Value	Meaning
1st (Minute)	`0`	At minute 0
2nd (Hour)	`3`	At 3 AM (03:00 24-hr format)
3rd (Day of Month)	`*`	Any day of the month
4th (Month)	`*`	Every month (1–12)
5th (Day of Week)	`0`	Sunday (`0` or `7` = Sunday)

8. Commands Used & Explanations

- `md5sum`: Computes 32-character hexadecimal MD5 hashes. Used to verify byte-for-byte identical content regardless of file name.
- `sort -k1,1`: Orders checksum output lexicographically by hash so identical files appear consecutively.
- `uniq -w 32 -d / -D`: Evaluates only the first 32 characters (`-w 32`) representing the MD5 hash. `-d` outputs duplicate hashes; `-D` outputs all repeated lines.
- `find "$DATA" -maxdepth 1 -type f`: Discovers regular files in the directory without descending into subfolders or scanning `_duplicates/`.
- `mv`: Atomically moves redundant files to `_duplicates/` during isolation and back during undo.
- `du -sh`: Reports total disk usage in human-readable units (K, M, G).
- `tar -czf`: Compresses data directory into a `.tar.gz` archive (`-c` create, `-z` gzip, `-f` archive file).

9. What I Learned

1. **Modular Shell Architecture**: Breaking code into a controller (`main.sh`) and a library (`lib/functions.sh`) significantly improves code maintainability and testability.
2. **Power of Linux Pipelines**: Combining `find` , `md5sum` , `sort` , and `uniq -w 32` allows complex duplicate detection in pure POSIX utilities without needing external heavyweight packages.
3. **Defensive Systems Programming**: Implemented input validation (`[[ $ch =~ ^[0-8]$ ]]`), verified file existence before reading (`check_data`), maintained an undo manifest (`logs/moved_files.txt`), and logged all operations.
4. **Automated Scheduling**: Learned user-level crontab scheduling (`crontab -e`), stderr/stdout redirection (`2>&1`), and headless CLI automation (`--cron`).
5. **Safe Data Lifecycle**: Discovered the value of non-destructive duplicate isolation (relocation instead of instant deletion) paired with transactional rollback.